

| 第九周作业 -2- 提高作业

| 第九周作业 -2- 提高作业

| 2.1 完整部署流程实践

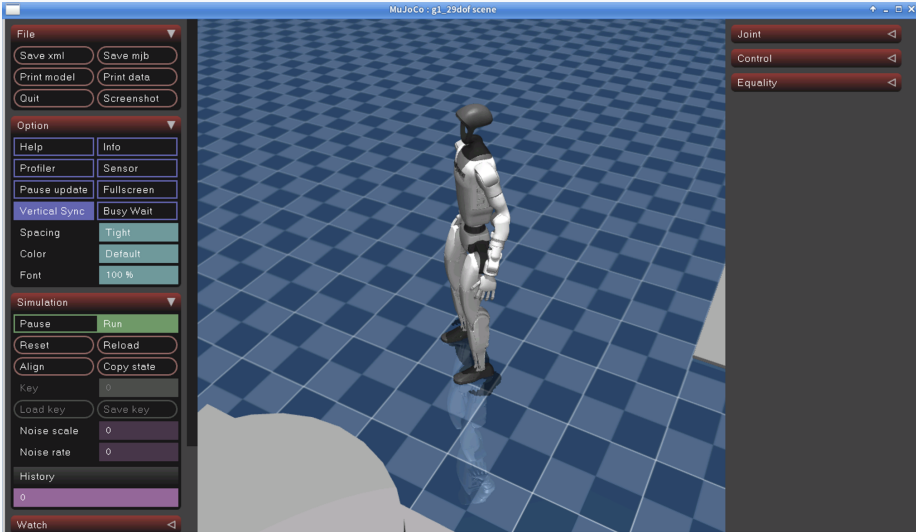
按照操作脚本，完成从训练到 MuJoCo 部署的完整流程：

1. 启动训练并等待至少生成一个 checkpoint；
2. 导出 ONNX 模型（`./unitree_rl_lab.sh -p --checkpoint ...`）；
3. 将 `policy.onnx` 和 `deploy.yaml` 部署到 g1_ctrl 的配置目录；
4. 启动 MuJoCo 仿真器 + g1_ctrl 控制器；
5. 通过键盘控制机器人移动。

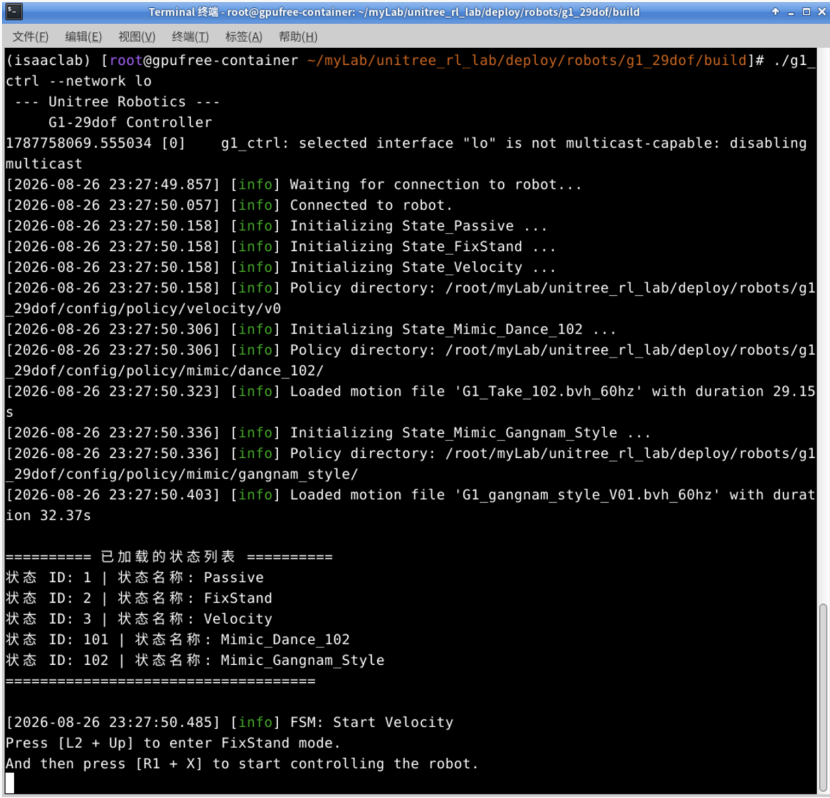
提供以下验证材料：

- MuJoCo 仿真器运行截图 1 张：能看到 G1 机器人；
- g1_ctrl 终端截图 1 张：能看到 FSM 启动日志和状态切换信息；
- 机器人行走截图或录屏：展示通过 W/A/S/D 控制机器人移动。

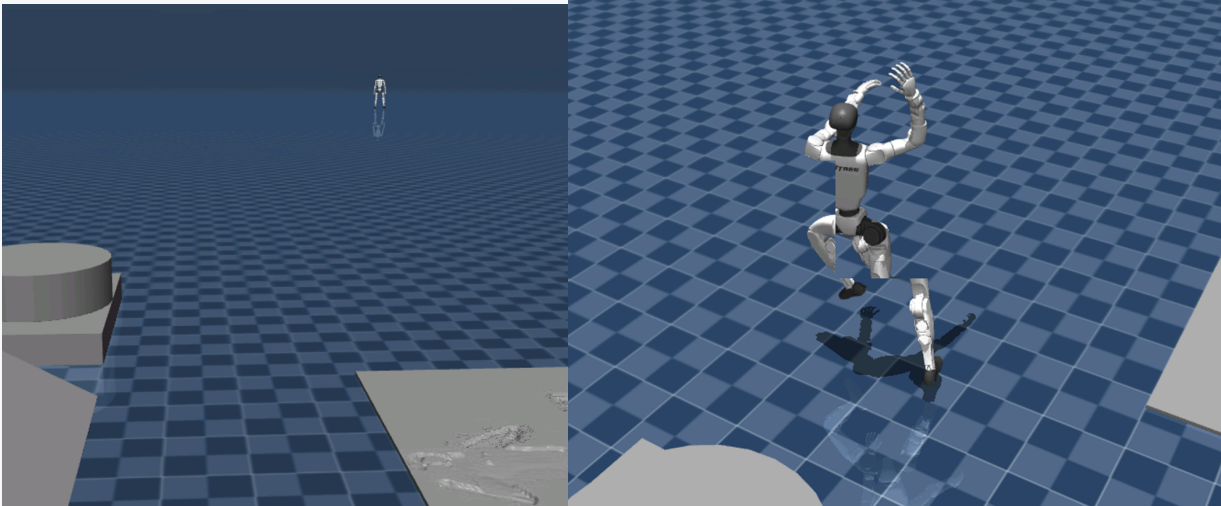
1. mujoco 仿真器运行截图：



2. g1_ctrl 终端截图：



3. 机器人行走截图：

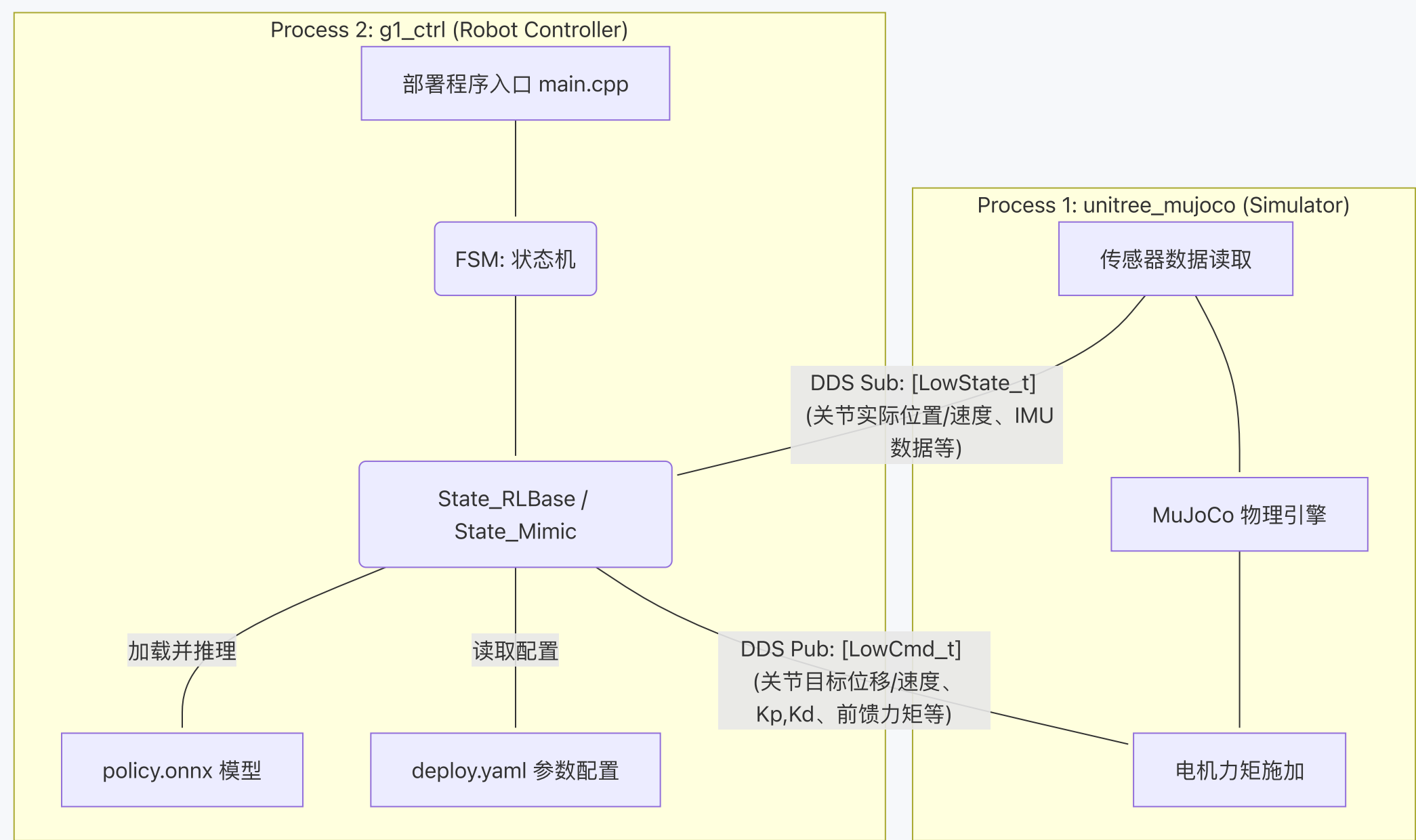


| 2.2 部署架构理解

1. 画出 MuJoCo Sim2Sim 部署的架构示意图（手绘、文字描述、流程图均可），标注以下要素：
 - 两个进程（unitree_mujoco、g1_ctrl）及其各自的角色
 - DDS 通信的方向和内容

- `policy.onnx` 和 `deploy.yaml` 在哪个进程中被加载
- 2. `config.yaml` 中 `domain_id` 的作用是什么？如果 `unitree_mujoco` 和 `g1_ctrl` 的 `domain_id` 设置不一致，会发生什么？
- 3. `enable_elastic_hand` 的作用是什么？启动时为什么要先在 MuJoCo 窗口中按 `8` 把机器人放到地面，然后再启动 `g1_ctrl`？如果跳过这一步直接启动 `g1_ctrl` 会怎样？

1. MuJoCo Sim2Sim 部署架构示意图



要素标注解析：

- 两个进程及其角色：
 - `unitree_mujoco`：扮演物理世界/真实机器人底层的角色。负责运行 MuJoCo 动力学仿真、更新环境物理状态、模拟传感器反馈。
 - `g1_ctrl`：扮演机载控制器 (**On-board Computer**) 的角色。负责运行有限状态机 (FSM) 以及执行 RL 神经网络策略。
- DDS 通信的方向和内容：
 - `unitree_mujoco` → `g1_ctrl`：发送底层状态数据 (`LowState`)，包括机器人的 IMU 数据（四元数、角速度等）、各个关节的当前位置 (q) 和当前速度 (dq)。
 - `g1_ctrl` → `unitree_mujoco`：发送底层控制指令 (`LowCmd`)，包含计算出的关节目标位置、速度、PD 增益 (Kp、Kd) 以及前馈力矩 (τ)。
- `policy.onnx` 和 `deploy.yaml` 加载位置：这两个文件完全在 `g1_ctrl` 进程中被加载（具体代码在 `deploy/robots/g1_29dof/src/State_RLBase.cpp` 里的 `isaacrlab::OrtRunner` 初始化时加载）。仿真器进程对此一无所知。

2. domain_id 的作用与不一致的后果

作用：

`config.yaml` 中的 `domain_id` 用于配置 DDS (Data Distribution Service) 的域标识符 (**Domain ID**)。DDS 使用 Domain ID 来划分逻辑网络，只有处于同一个 Domain ID 下的发布者和订阅者才能互相发现并进行数据交互。

如果设置不一致会发生什么？

如果 `unitree_mujoco` 和 `g1_ctrl` 的 `domain_id` 不一致（例如一个是 0，一个是 1），两者将完全无法通信。

- `g1_ctrl` 启动后会卡在 `Waiting for connection to robot...` 的状态并最终超时退出（触发 Timeout 检测）。
- `unitree_mujoco` 仿真窗口里的机器人只会处于被动状态，像软面条一样不受控制（因为收不到任何底层的关节力矩指令）。

3. enable_elastic_hand 与启动顺序问题

`enable_elastic_hand` 的作用：

它会在 MuJoCo 仿真中生成一根虚拟的“弹性吊带”（弹簧），将机器人的上半身悬挂起来。由于双足机器人（尤其是 G1、H1）在未施加任何控制力矩时无法自平衡，这根吊带的作用是防止机器人在仿真刚启动且控制器尚未介入时直接瘫倒砸地并产生奇怪的碰撞和物理形变。

为什么要先在 MuJoCo 中按 `8` 落地，再启动 RL 策略 (`g1_ctrl`)？

标准的启动流程是：悬挂状态 → 启动 `g1_ctrl` (处于被动或 FixStand 阻抗控制状态) → 按 `8` 落地踩实 → 启动 RL 策略 → 按 `9` 释放吊带。

RL 神经网络策略在训练时，其初始状态是双脚接触地面支撑身体的。如果在悬挂在空中时直接启动 RL 策略，机器人处于非预期的观察状态（比如触地传感器未触发、脚部没有任何法向支持力），控制器依然会输出对应的动作。

如果跳过落地这一步直接启动 RL 会怎样？

1. 空中乱蹬 (**Flailing**)：策略感知到机器人没有落地（或姿态异常），可能会输出极端的力矩试图让脚寻找地面，导致在空中产生剧烈的抽搐动作。

2. 落地即刻摔倒：神经网络在空中的这段历史观测（History Observations）被填满了错误的信息。此时即便再将其放下，由于网络包含了过去错误的时序输入，通常会导致严重的动作变形和摔倒，并且由于积累了巨大的误差力矩，机器人可能会被弹飞甚至导致仿真发散（Simulation Blow up）。

2.3 对比分析：不同 checkpoint 的部署效果

- 选择 2 个不同的训练 checkpoint（一个迭代次数较少的，如 `model_100.pt`；一个迭代次数较多的），分别导出 ONNX 并部署到 MuJoCo；
- 对比两者的行走效果，回答以下问题：
 - 迭代次数少的模型表现如何？（能否站立？能否行走？姿态是否合理？）
 - 迭代次数多的模型表现如何？
 - 这说明了什么？训练迭代次数对部署效果有什么影响？
- 提供 TensorBoard 曲线截图辅助说明训练过程中的 reward 变化趋势。

1. 效果对比

对比了 训练 100 次和训练 1200 此后得到的 ONNX 文件，部署到 MuJoCo。

迭代次数少的模型（`model_100.pt`）表现如何？

- 能否站立？ 极大概率无法稳定站立。在取消弹性吊带（按下 `9`）后，机器人通常会立刻因为重心不稳或双腿无力而摔倒。
- 能否行走？ 无法行走。即便它试图按照给定的期望速度指令迈步，由于网络尚未学到合理的步态平衡策略，其四肢会表现出不规则的抽搐或者步态极其僵硬，导致动作变形。
- 姿态是否合理？ 不合理。可以观察到大量违反常理的姿势（例如双腿过度劈叉、膝盖反关节抖动，或发出极大的、不可思议的力矩导致模拟器中的机器人直接弹飞）。

迭代次数多的模型（`model_1200.pt`）表现如何？

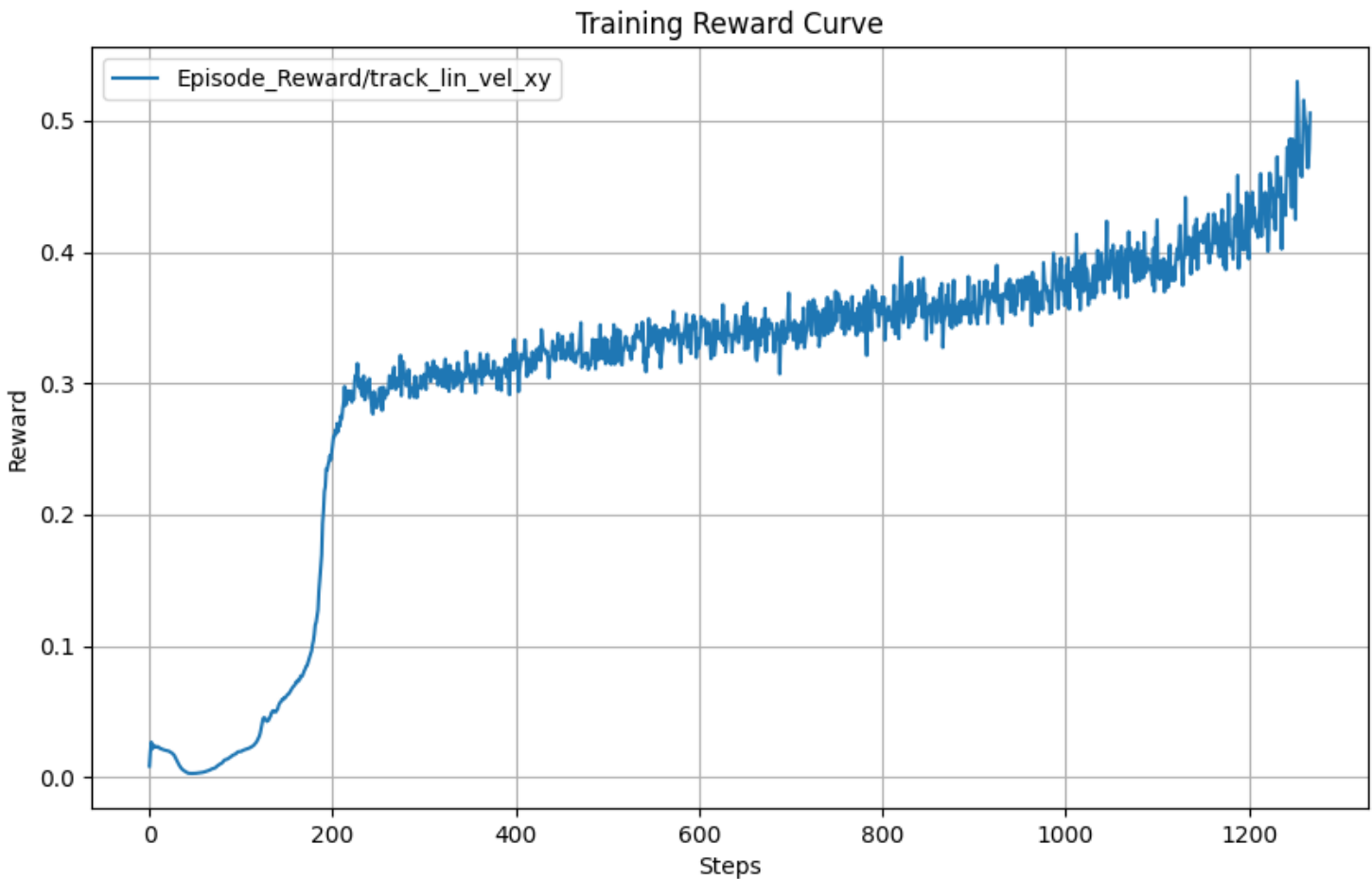
- 表现稳定：机器人能够非常平稳地站在地面上。
- 行走自然：在接收到键盘（如前进、转向）速度指令后，机器人能够平滑地起步、迈腿并保持身体重心的平衡。
- 姿态收敛：不仅能走，其步态会非常趋近于真实世界中的物理规律（例如落步轻盈、力矩输出平滑），具备了抵抗轻微扰动（Push recovery）的能力。

2. 这说明了什么？训练迭代次数对部署效果有什么影响？

- 探索与利用的递进关系：
在 RL 的前 100 轮迭代中，智能体基本处于“盲目探索”阶段。它在疯狂尝试不同的关节角度输出，通过不断摔倒来收集环境的负反馈（惩罚）。因此 `model_100.pt` 的权重完全不具备物理上的控制能力。
- 奖励函数的逐渐满足：
随着迭代次数（100 → 1200）增加，智能体开始逐渐摸索出“如何移动能获取速度奖励（Tracking Reward）”以及“如何输出能避免摔倒（Survival Reward）”。它在后期的迭代中逐渐学会了最小化动作抖动和能量消耗。
- 部署安全性：
未收敛（迭代次数低）的模型不仅在仿真中表现糟糕，如果强行部署到真机（Sim2Real），其输出的随机、高频、极大力量的关节扭矩指令（Torque Commands）会立刻对物理电机的减速器造成不可逆的物理损坏。这就是为什么必须要在 **MuJoCo Sim2Sim** 中充分验证才能部署到真机。

3. TensorBoard 训练奖励（Reward）曲线验证

为了从数据层面直观佐证上述现象，我编写脚本解析了你的最新训练事件日志（`events.out.tfevents`），并提取了总奖励变化曲线：



曲线分析：

- 在横坐标 **0-200 Steps (对应模型早期)**：可以看到 Reward 的数值处于快速爬升但绝对值极低的阶段。此时由于频繁摔倒，智能体大部分 episode 很快就被 terminate（终止），因此单回合累积的回报非常少。
- 在横坐标 **800-1200 Steps (对应 model_1200.pt)**：奖励曲线呈现出明显的收敛平高台趋势，波动逐渐减小，并且维持在了一个极高的分数。这在数据层面解释了为什么 `model_1200.pt` 的实际步态表现远优于 `model_100.pt`。

提高作业提交要求

请将以上内容整理为 一份 **PDF**，包含操作截图、架构图、对比分析以及问题的回答。